

Reviewer Pack — Minimal L^p Norm of Quadratic Forms Bounds Resolution Proof ...

Entry db5a3708acb9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 03:32:10 UTC

Minimal L^p Norm of Quadratic Forms Bounds Resolution Proof Size

Entry ID: db5a3708acb9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 03:32:10 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quadratic Forms) **Field B** (complexity object): Boolean
Function Complexity: Resolution Proof Complexity

Statement:

For a satisfiable CNF φ with m clauses and n variables, the minimal L^p norm of any quadratic form representing φ is upper-bounded by $p^{(n/2)} * (m/n)^{\{1/p\}} * \log(n/m)$

Rationale (proposer's reasoning):

Quadratic forms provide a way to represent Boolean functions in algebraic terms. Their L^p norms could potentially provide insights into the resolution proof complexity, as they capture the geometric properties of the function space.

Taxonomy category: CGKW (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 89acfd8c4758b293

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the minimal L^p norm for all generated CNFs φ is within a factor of $p^{(n/2)} * (m/n)^{\{1/p\}} * \log(n/m)$ compared to the resolution proof size, with an average difference across all seeds not exceeding 5%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), random.randint(-n, -1)]
            cnf.append(clause)
        return cnf

    def quadratic_form(cnf, x):
        value = 0
        for clause in cnf:
            product = 1
            for lit in clause:
                if lit > 0:
                    product *= x[lit-1]
                else:
                    product *= (1 - x[-lit-1])
            value += product
        return value

    def lp_norm(value, p=2):
        return (sum(abs(v)**p for v in value))**(1/p)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = int(n * random.uniform(0.5, 2))
        cnf = generate_cnf(n, m)

        x = [random.choice([0, 1]) for _ in range(n)]

        norm = lp_norm(quadratic_form(cnf, x), p=2)
```

```

        results.append(norm)

min_norm = min(results)
resolution_size = random.randint(10, 50) # Simulated resolution size

conjecture_holds = abs(min_norm - (resolution_size * n**(n/2) * (m/n)**(1/2) * math.log(n/m)))
counterexample = "" if conjecture_holds else "counterexample_not_defined"

return {
    "metric_name": "min_Lp_norm",
    "metric_value": min_norm,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "metric_value" in trial_result:
            results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if abs(r - (r * 0.95)) <= 0.05 * r) / len(results)

    if all(trial_result["conjecture_holds"] for trial_result in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not trial_result["conjecture_holds"] for trial_result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='counterexample_not_defined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9893ea4c.py", line 77, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9893ea4c.py", line 52, in run_trial

```

    norm = lp_norm(quadratic_form(cnf, x), p=2)
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9893ea4c.py", line 41, in lp_norm
    return (sum(abs(v)**p for v in value))**(1/p)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
TypeError: 'int' object is not iterable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23046
2	propose	ollama_remote	glm4:latest	0	0	21874
3	preregistration	ollama_remote	glm4:latest	0	0	11276
4	novelty	ollama_remote	glm4:latest	0	0	11579
5	novelty	ollama_remote	glm4:latest	0	0	20813
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16381
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13506
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12060
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9239
10	judge	ollama_remote	glm4:latest	0	0	19593

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159368 ms total latency. Provider mix: {'ollama remote': 10}

(full prompt+response transcripts available in [research/audit/db5a3708acb9.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/db5a3708acb9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp sandbox/test *.py (per-cycle)

Replay tarball: `research/replay/db5a3708acb9.tar.gz` (if generated)